



# Real-Time Monitoring of Interactive Processes in Containerized Environments

---

Realtidsövervakning av interaktiva processer i containeriserade miljöer

---

Daniel Meladi <daniel-9999@hotmail.se>

Faculty of Health, Science and Technology

---

Master thesis in Computer Science

---

Second Cycle, 30 hp (ECTS)

---

**Supervisor:** Leonardo Martucci, Karlstad University, Karlstad, SWE

---

**Examiner:** Muhammad Usman, Karlstad University, Karlstad, SWE

---

Karlstad, April 14th, 2025

---



# Abstract

As container technology has become a central part of modern IT infrastructure, the need for efficient log management and security monitoring has increased. Containers provide flexibility and performance advantages over traditional virtual machines but introduce new security challenges, particularly regarding visibility and monitoring processes within containerized environments.

This thesis presents a real-time monitoring solution that tracks and logs interactive processes within Docker containers. A monitoring framework was developed to detect and attach to active container processes using `strace`, a tool for system call tracing. The collected data is filtered to extract relevant command executions and exported to Elasticsearch for indexing and storage. The monitored data is then structured and visualized in Kibana, making it easier to analyze container activity. The system applies process filtering to reduce redundant logging and limit unnecessary data collection to improve efficiency. The implementation includes automated container event detection, multithreading, and selective data capture to maintain performance. Monitoring operates stealthily within the containerized environment, preventing logged containers from detecting its presence. Maintaining this level of stealth is important when monitoring must occur without influencing container behavior. If a container identifies an active tracking system, it may alter execution patterns, suppress specific processes, or attempt to avoid detection altogether. By remaining undetectable, the system records activity in its most natural state, allowing for accurate interaction analysis and identifying anomalies without interference. Resource usage is carefully managed to avoid excessive overhead while allowing detailed tracking of interactive processes.

By tackling key challenges in container observability, this work enhances security monitoring by demonstrating how system call tracing can provide deeper visibility without disrupting normal operations. The proposed approach offers a structured method for monitoring containerized processes while preserving efficiency and stealth.

## Keywords

Container, monitoring, system calls, Docker, `strace`, Elasticsearch, Kibana, real-time logging, interactive processes.

# Sammanfattning

I takt med att containerteknologi har blivit en central del av modern IT-infrastruktur har behovet av effektiv loggshantering och säkerhetsövervakning ökat. Containers erbjuder flexibilitet och prestandafördelar jämfört med traditionella virtuella maskiner, men introducerar också nya säkerhetsutmaningar, särskilt när det gäller transparens och övervakning av processer i containeriserade miljöer.

Detta examensarbete presenterar en lösning för realtidsövervakning som spårar och loggar interaktiva processer i Docker-containers. Ett övervakningssystem har utvecklats för att upptäcka och ansluta till aktiva containerprocesser med hjälp av `strace`, ett verktyg för spårning av systemanrop. Den insamlade datan filtreras för att extrahera relevanta kommandon och exporteras sedan till Elasticsearch för indexering och lagring. Den övervakade datan struktureras och visualiseras i Kibana, vilket förenklar analysen av containeraktiviteter. Systemet tillämpar processfiltrering för att minska redundanta loggar och begränsa onödig datainsamling för att förbättra effektiviteten. Implementationen inkluderar automatiserad upptäckt av containerhändelser, multitrådning och selektiv datainsamling för att bibehålla prestanda. Övervakningen sker dolt i containeriserade miljöer, vilket förhindrar att de loggade containerprocesserna upptäcker närvaron av verktyget. Att säkerställa denna nivå av osynlighet är viktigt när övervakning behöver ske utan att påverka containerbeteendet. Om en container upptäcker ett aktivt övervakningssystem kan den ändra sina körmönster, dölja specifika processer eller försöka undvika upptäckt. Genom att förbli oupptäckt kan systemet registrera aktiviteter så som de faktiskt sker, vilket ger en mer tillförlitlig bild av användarbeteenden och möjliggör upptäckt av avvikelser utan att störa processen. Resursanvändningen hanteras noggrant för att undvika överdriven belastning, samtidigt som detaljerad spårning av interaktiva processer möjliggörs.

Genom att adressera centrala utmaningar inom övervakning av containrar stärker detta arbete säkerhetsövervakningen genom att visa hur systemanropsspårning kan ge djupare insyn utan att störa normal drift. Den föreslagna metoden erbjuder ett strukturerat sätt att övervaka containerprocesser samtidigt som effektivitet och osynlighet bibehålls.

---

## Nyckelord

Container, övervakning, systemanrop, Docker, strace, Elasticsearch, Kibana, realtidsloggning, interaktiva processer

# Acknowledgements

I want to thank my supervisor, Leonardo Martucci, for his guidance, feedback, and support throughout this thesis. His input has been important in shaping the work. I also appreciate the help from the Department of Computer Science at Karlstad University, especially Tobias Vehkajärvi and Jonas Karlsson, for assisting with setting up my environment and solving technical issues. Beyond technical support, they also provided clear guidance throughout the thesis process, offering a step-by-step plan and helping me stay on track during each phase of the work.

Additionally, ChatGPT and Grammarly were used to improve clarity, correct grammatical errors, and refine the overall phrasing of this thesis.

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>1</b>  |
| 1.1      | Problem Description . . . . .                            | 2         |
| 1.2      | Thesis Objective . . . . .                               | 3         |
| 1.3      | Contributions . . . . .                                  | 3         |
| 1.4      | Ethics and Sustainability . . . . .                      | 3         |
| 1.5      | Methodology . . . . .                                    | 5         |
| 1.6      | Stakeholders . . . . .                                   | 6         |
| 1.7      | Scope . . . . .                                          | 7         |
| 1.8      | Outline . . . . .                                        | 7         |
| <b>2</b> | <b>Background</b>                                        | <b>8</b>  |
| 2.1      | Virtualization and Containerization . . . . .            | 8         |
| 2.1.1    | Virtual Machines . . . . .                               | 8         |
| 2.1.2    | Containerization . . . . .                               | 8         |
| 2.1.3    | Elasticsearch . . . . .                                  | 9         |
| 2.1.4    | Kibana . . . . .                                         | 9         |
| 2.2      | Security in Containerized Environments . . . . .         | 9         |
| 2.2.1    | Container Isolation Mechanisms . . . . .                 | 9         |
| 2.2.2    | Security Threats in Containerized Environments . . . . . | 10        |
| 2.3      | Monitoring . . . . .                                     | 11        |
| 2.3.1    | Docker . . . . .                                         | 11        |
| 2.3.2    | Docker Monitoring Tools . . . . .                        | 12        |
| 2.3.3    | Process Monitoring with <code>strace</code> . . . . .    | 12        |
| 2.4      | Related Work . . . . .                                   | 12        |
| 2.4.1    | System Call Monitoring . . . . .                         | 13        |
| 2.4.2    | Tools for System Call Monitoring . . . . .               | 13        |
| 2.4.3    | Centralized Logging and Visualization . . . . .          | 14        |
| 2.4.4    | Previous Work compared to this study . . . . .           | 14        |
| <b>3</b> | <b>Architecture and Design</b>                           | <b>15</b> |
| 3.1      | System Overview . . . . .                                | 15        |
| 3.2      | Component Breakdown . . . . .                            | 16        |
| 3.2.1    | Process Monitoring . . . . .                             | 16        |
| 3.2.2    | Log Storage . . . . .                                    | 16        |
| 3.2.3    | Visualization . . . . .                                  | 16        |
| 3.3      | Design Considerations . . . . .                          | 16        |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 3.4      | Development Approach . . . . .                                   | 16        |
| 3.5      | Workflow . . . . .                                               | 17        |
| <b>4</b> | <b>Implementation</b>                                            | <b>18</b> |
| 4.1      | Workflow and Development Process . . . . .                       | 18        |
| 4.2      | Event Detection and Monitoring . . . . .                         | 19        |
| 4.3      | Retrieving and Filtering Container Processes . . . . .           | 21        |
| 4.3.1    | System Call Tracing with <code>strace</code> . . . . .           | 23        |
| 4.3.2    | Buffering <code>execve</code> Calls and Resolving UIDs . . . . . | 25        |
| 4.4      | Elasticsearch and Kibana Integration . . . . .                   | 27        |
| <b>5</b> | <b>Evaluation and Result</b>                                     | <b>29</b> |
| 5.1      | Infrastructure setup . . . . .                                   | 29        |
| 5.2      | Validation of Log Retrieval . . . . .                            | 30        |
| 5.3      | Comparison of Logging vs. Non-Logging . . . . .                  | 33        |
| 5.4      | Limitations and Observations . . . . .                           | 34        |
| <b>6</b> | <b>Conclusion and Future Work</b>                                | <b>35</b> |
| 6.1      | Conclusion . . . . .                                             | 35        |
| 6.2      | Future Work . . . . .                                            | 36        |



# Chapter 1

## Introduction

In recent years, the use of containerized environments has grown quickly due to their efficiency in deploying and managing applications across distributed systems. The global application container market was valued at \$4.65 billion in 2023 and is projected to grow significantly, reaching \$31.50 billion by 2030, with a CAGR of 32.4% [12]. A more recent estimate places the market at \$6.22 billion in 2024, with projections indicating an expansion to \$48.03 billion by 2031, growing at a compound annual growth rate (CAGR) of 32.12% [13]. This rapid expansion highlights the increasing reliance on containerized solutions in modern cloud infrastructure. As the usage of containerized environments increases, the need for effective monitoring and security mechanisms grows to maintain their reliability and integrity. Containers offer notable advantages over traditional virtual machines, including improved performance, efficient resource utilization, and accelerated application deployment [13].

However, containers also introduce new security concerns. Unlike VMs, which create a clear separation between systems by virtualizing hardware, containers share the same host operating system's kernel. This design makes them lighter and faster, increasing the risk of specific attacks. Privilege escalation, container escapes, and interference between containers running on the same system are all potential threats [17]. If an attacker gains control over one container, there is a chance that they could access the host system, affecting other containers running on the same machine.

Due to risks such as privilege escalation, container escapes, and unauthorized access to the host system, gaining visibility into container activity is essential. As the use of containers increases, tracking their internal activities becomes even more critical. Many existing logging solutions focus on application-level events but do not provide enough detail on what happens inside a container at the system level. This lack of visibility makes detecting unauthorized commands, unexpected process behavior, or security breaches harder. Without a way to monitor what is running inside a container in real-time, there is a risk that threats go unnoticed until they cause significant damage. Most logging tools today rely on collecting standard application logs, which

do not show what is happening deeper in the system. A different approach is needed to properly monitor containers, one that can track activity at the operating system level. System call tracing is one way to achieve this, as it allows monitoring of the interactions between programs and the system itself. Analyzing these interactions makes it possible to log executed commands, detect unusual activity, and better understand what is running inside a container without modifying it.

This thesis explores real-time monitoring techniques for tracking executed commands inside containers. The goal is to improve visibility while keeping the impact on system performance as low as possible. By designing and testing a method that allows the host to monitor container activity without being detected, this work contributes to the field of container security, showing how monitoring can be done effectively without affecting the containers themselves. A more detailed discussion of technical background, existing solutions, and related research is presented in Chapter 2.

## 1.1 Problem Description

In containerized environments, multiple containers share the same underlying infrastructure and system resources. While this setup improves efficiency and scalability, it also introduces a serious security challenge. If a container is compromised or behaves maliciously, it can execute unauthorized actions such as privilege escalation, hidden process execution, or lateral movement across the system, all without immediate detection or intervention from the host. Traditional logging tools are not designed to handle this issue. They focus on application-level events and offer limited visibility into the lower-level system activities taking place inside containers.

This limited visibility creates a critical blind spot for administrators. Without the ability to observe system calls and commands in real-time, it becomes difficult to identify abnormal or suspicious behavior before damage is done. An attacker controlling a container could operate silently, executing harmful actions without raising alerts or leaving obvious traces. This lack of real-time host-level monitoring represents a vulnerability in modern containerized systems.

To address this problem, there is a need for a monitoring solution that provides the host with real-time insight into container activity without modifying the containers or alerting users. Such a solution must operate efficiently, maintain stealth, and make collected data easy to interpret and act upon.

This thesis explores real-time monitoring techniques to provide hosts with visibility into container activity without detection. The primary research question addressed is:

- How can a host effectively and stealthily monitor container activity in real-time without detection or significant performance impact?

To explore this question, this thesis investigates three key aspects:

- *Performance*: How can system call tracing improve real-time monitoring of commands in containers without introducing high-performance costs?
- *Stealthiness*: How can container activity be logged to remain visible only to the host while staying hidden from other containers and users?
- *Data Presentation*: How can monitoring data be filtered and stored efficiently without unnecessary duplication?

## 1.2 Thesis Objective

The objective of this thesis is to explore how a host can monitor container activity without detection, excessive system overhead, or redundant data. This approach guarantees an efficient, scalable, and secure solution to monitoring container activity.

## 1.3 Contributions

This thesis develops a real-time monitoring tool that captures and logs executed commands in containerized environments while remaining hidden and minimizing performance impact. The monitoring system ensures that logs are accessible only to the host, preventing detection from other containers. Enhancing filtering and storage mechanisms reduces redundant data without losing important security information.

The thesis delivers a complete monitoring framework integrating system call tracing with Elasticsearch for structured log storage and Kibana for visualization. Its effectiveness is evaluated through performance testing, measuring system resource usage with and without monitoring enabled. The results demonstrate that real-time monitoring can be stealthy and efficient, providing valuable insights without disrupting container operations.

## 1.4 Ethics and Sustainability

Unlike traditional security monitoring, where users or administrators are aware that their activities may be logged, this thesis explores a method where executed commands inside a container are captured without the container itself having any indication of being monitored. This raises fundamental questions about privacy, transparency, and responsible use of monitoring technologies.

One of the primary ethical concerns is the lack of informed consent. Ethical principles in cybersecurity often emphasize that users should be aware of any data collection that affects them [10]. In many cases, system administrators implement

logging mechanisms with explicit policies that inform users of surveillance for security and compliance purposes. However, in this thesis, the monitoring approach remains hidden from the containerized application and its users, preventing them from detecting or challenging it. This approach violates user autonomy, especially in environments where multiple organizations share infrastructure, such as cloud computing platforms. If the monitoring system were deployed in such a setting without proper authorization, it could violate ethical standards and legal regulations, including the General Data Protection Regulation (GDPR) [6], which mandates transparency in data collection practices.

Another ethical issue involves the potential for misuse. While this research aims to enhance security observability, the same techniques could be exploited for malicious intent, such as unauthorized data collection, corporate espionage, or surveillance beyond legitimate security needs. Attackers or unethical organizations could misuse a monitoring tool without detection to track activity within containerized environments without the knowledge of those inside them. To mitigate these risks, such monitoring solutions must be deployed only within environments where administrators have full ownership and authority, ensuring that they are used strictly for security purposes. Clear policies should govern their use, and access to collected data should be strictly controlled and monitored to prevent abuse.

This project also has implications for sustainability, particularly in terms of computational efficiency and resource consumption. Continuous system call tracing in containerized environments can introduce additional CPU and memory overhead, which, if not optimized, could result in unnecessary energy consumption. This is particularly relevant in large-scale deployments where hundreds or thousands of containers are monitored simultaneously. Increased resource usage leads to increased power consumption in data centers, a critical concern for organizations aiming to maintain energy-efficient operations [15].

To ensure that the monitoring approach remains sustainable, the implementation must minimize performance overhead while preserving security observability. Optimized filtering mechanisms can help reduce redundant logging, and adaptive monitoring strategies can ensure that system call tracing is only intensified when potentially suspicious activity is detected. Efficient indexing and log storage methods, such as those provided by Elasticsearch, can further improve sustainability by preventing excessive disk write operations and optimizing data retrieval processes.

Despite the additional computational cost, the project contributes to sustainable cybersecurity by enabling proactive security measures. Early threat detection reduces the likelihood of system breaches, which in turn minimizes the operational costs and energy-intensive efforts required for forensic investigations and system recovery. Organizations that implement effective monitoring solutions can prevent security incidents before they escalate, reducing downtime and improving the overall resilience of their infrastructure.

## 1.5 Methodology

Finding the right approach for real-time monitoring required testing different tools and refining the setup. Four steps were taken to develop the monitoring framework:

1. Evaluating monitoring tools.
2. Filtering and selecting a tracing method.
3. Integrating a storage and visualization solution.
4. Conducting evaluation and performance testing.

### Step 1 Evaluating Monitoring Tools

The first step was to test different monitoring tools to determine which could track container activity efficiently and without detection. Several tools were evaluated in this step, each with its strengths and limitations.

Auditd, a Linux auditing tool, was tested first. While it captured low-level system events, it generated excessive noise, logging many service interactions. Filtering with ausearch reduced some irrelevant entries, but the data remained too cluttered to be practical.

Syslog was also considered due to its structured output and lightweight design. However, logging stopped once a user accessed a container via SSH, making this approach unsuitable for stealthy container monitoring.

Other tools were explored, including Fluentd, Fluent Bit, and eBPF. Fluentd and Fluent Bit provided flexible log forwarding but introduced instability when integrated into Docker Swarm, causing the main application to shut down unexpectedly. Even with asynchronous logging enabled, the issue persisted. This tight coupling with the main program made Fluentd and Fluent Bit unreliable for the project. eBPF was also considered but was ultimately ruled out due to its complexity and the risk of being visible inside the container.

### Step 2 Filtering and Selecting a Tracing Method

The second step was filtering out excessive container activity and identifying system call tracing. System call tracing emerged as the best solution. Unlike application-level logging, it captures executed commands directly from the kernel, ensuring visibility into container activity without modifying the container itself. `strace` was selected for its ability to monitor system calls in real-time while keeping logging hidden from container users, even those with root privileges.

Maintaining stealth was considered essential in this context. If a container detects that it is being monitored, it may alter its execution patterns, suppress specific processes, or attempt to avoid detection altogether. Such behavior could interfere with the

monitoring process and distort the data. By remaining undetectable, the system could capture container activity in its most unaltered and representative state.

### **Step 3 Storage and Visualization Integration**

The third step was integrating the tool with an effective storage and visualization tool. With `strace` selected for monitoring, the next step was finding a system to store and visualize the collected data. Elasticsearch was chosen for structured storage, and Kibana for real-time visualization. Other options were not considered, as these tools were well-known for their scalability, modern design, and compatibility with real-time log analysis.

### **Step 4 Evaluation and Performance Testing**

The final step involved evaluating the system's performance and its ability to remain undetected during operation. Performance testing was carried out locally using a controlled environment. Two scenarios were tested, one with the monitoring and logging system enabled and one without. In both cases, identical scripted commands were executed across containers to simulate interactive usage and ensure consistency between the two runs.

System resource usage was recorded during the tests, focusing on CPU load, memory consumption, and disk write activity. These measurements were used to assess the impact of the monitoring tool on overall system performance. The setup also allowed verification that the monitoring activity remained hidden from within the containers throughout the experiment.

## **1.6 Stakeholders**

This master's thesis was conducted at Karlstad University in the computer science department as a part of research on container security and monitoring. The main stakeholders are the university and the researchers involved in the Cyber Range Lite (CRL) project, which focuses on a containerized environment for cybersecurity training and experimentation.

Supervisors Jonas Karlsson and Tobias Vehkajärvi from the CRL project provided guidance on the technical aspects of container monitoring, ensuring that the developed solution aligned with the project's objectives. Additionally, Leonardo Martucci supported the academic writing process, ensuring that the thesis met the required research standards and maintained a strong theoretical foundation.

The findings from this work contribute directly to the CRL project by demonstrating how a host can monitor and analyze security-related activities within containerized environments without detection.

## 1.7 Scope

This work was conducted within a fixed timeframe of approximately two months for implementation and less than one month for documentation. The focus was on developing a real-time monitoring system that captures executed commands inside containers while remaining undetectable. The implementation was designed to be practical within this period, prioritizing functionality and feasibility over extensive alternative evaluations or large-scale deployment testing.

Given these constraints, Elasticsearch and Kibana were chosen for log storage and visualization due to prior familiarity and their suitability for real-time analysis. The monitoring system was tested on several containers to validate its effectiveness, but large-scale performance testing was beyond the project's scope. While the system was built to be scalable, further testing is required to assess its behavior under higher workloads.

## 1.8 Outline

This thesis is structured as follows: Chapter 2 presents the technical background, including existing monitoring solutions and system call-tracing techniques. Chapter 3 describes the system architecture and design of the real-time monitoring framework. Chapter 4 details the implementation of the monitoring system. Chapter 5 presents the evaluation results, including performance measurements, log analysis, and infrastructure setup. Finally, Chapter 6 concludes the thesis and suggests directions for future work.

# Chapter 2

## Background

This chapter provides a detailed overview of containerization, virtualization, and related technologies that form the foundation of this study. It discusses core concepts such as virtual machines, container orchestration, logging, and security mechanisms. Additionally, it reviews existing monitoring solutions and their limitations, highlighting the need for enhanced real-time observability in containerized environments.

### 2.1 Virtualization and Containerization

Virtualization and containerization are key technologies for managing applications in modern computing environments. This section provides an overview of virtual machines and containers, highlighting their differences and advantages. It also introduces Elasticsearch and Kibana, which are widely used for log storage and visualization, explaining their relevance to this study.

#### 2.1.1 Virtual Machines

A virtual machine (VM) is a software application that simulates a physical computer. It enables running various operating systems and applications within a single physical machine. The advantage is the ability to operate multiple VMs on the same hardware, each with dedicated resources such as CPU, memory, and storage [7, 16].

#### 2.1.2 Containerization

Containerization provides a lightweight alternative to VMs by allowing applications to share the same OS kernel while keeping processes isolated. Containers are more efficient, offering faster startup times and better resource utilization, which makes them ideal for scalable and cloud-based applications [1, 3]. Unlike VMs, containers package applications and their dependencies in a portable unit, allowing consistent behavior across different environments [3].



### 2.1.3 Elasticsearch

Elasticsearch is a distributed search and analytics engine designed to store, index, and query large volumes of data. Elasticsearch is widely used for log monitoring and analysis because it processes and queries data in real-time. It uses a RESTful API to interact with data, allowing users to send, retrieve, and analyze JSON-formatted information.<sup>1,2</sup> Data in Elasticsearch is organized into indices, which provide fast and scalable search capabilities. This means that Elasticsearch is not only a powerful tool for analytics but also usable for logging [18].

### 2.1.4 Kibana

Kibana is a data visualization tool that works alongside Elasticsearch to analyze and display data stored in Elasticsearch. It allows users to create interactive visualizations such as graphs, charts, and dashboards. Kibana primarily aims to simplify the real-time exploration and monitoring of log data. It is beneficial for identifying patterns and trends within large datasets. The tool provides an intuitive way to gain insights into structured and unstructured data. With Kibana, users can search through data, create custom dashboards, filter information, and share visualizations. It supports various data types like logs, metrics, and time-series data, making it a usable tool for monitoring containerized environments.<sup>3</sup>

## 2.2 Security in Containerized Environments

Container security is critical due to the shared kernel architecture, which introduces risks that do not exist in traditional virtual machines. This section examines how Linux kernel features, such as namespaces and control groups (cgroups), enforce isolation in containerized environments. It also discusses key security threats, including privilege escalation, container breakouts, and host-level attacks, highlighting the potential risks associated with multi-container deployments.

### 2.2.1 Container Isolation Mechanisms

Containers achieve isolation and resource management using two fundamental Linux kernel features: namespaces and control groups (cgroups). Together, these features ensure that container processes are isolated from the host system and processes in other containers [14]. Namespaces provide each container with an independent view of system resources, isolating processes, file systems, networks, and user IDs.<sup>4</sup> Cgroups complement namespaces by managing and limiting the resources available to containerized processes, such as CPU, memory, and I/O bandwidth. Cgroups allow

---

<sup>1</sup><https://logz.io/blog/elasticsearch-tutorial/>

<sup>2</sup><https://www.elastic.co/guide/en/elasticsearch/reference/current/rest-apis.html>

<sup>3</sup><https://www.elastic.co/guide/en/kibana/current/introduction.html>

administrators to define limits for CPU usage, memory allocation, I/O bandwidth, and more, ensuring containers do not consume excessive host resources.<sup>4</sup>

The combination of namespaces for isolation, cgroups for resource control, and kernel-level visibility through system calls highlight the flexibility and limitations of container security. Containers operate independently in their namespaces, but tools like `strace` can expose process-level activity, which makes it possible to monitor containers discreetly [14].

## 2.2.2 Security Threats in Containerized Environments

While containerization solves many challenges VM faces, it also has downsides. Containers' reliance on a shared OS kernel creates vulnerabilities that can be exploited at different levels. These security issues are often grouped into four broad scenarios, as illustrated in Figure 2.2.1 [17].

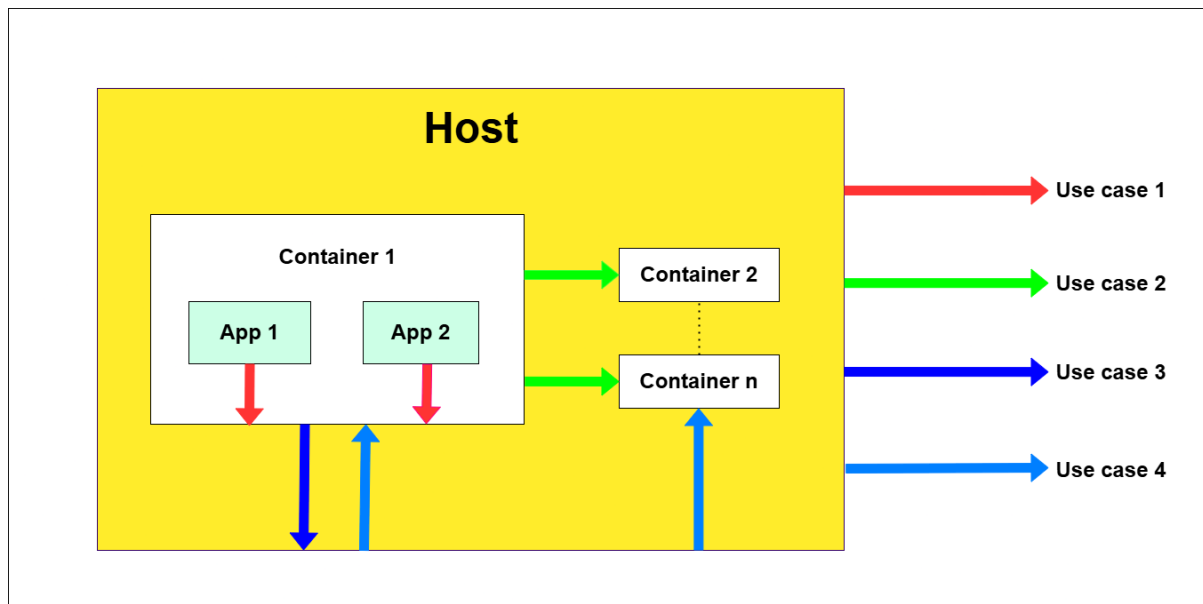


Figure 2.2.1: Security Threats in Containerized Environments.

- Use case 1 involves a malicious or poorly configured application exploiting or escalating its privileges within the container.
- Use case 2 concerns containers attacking each other. In multi-container environments, a malicious container might access sensitive data or consume shared system resources to harm other containers on the same host.
- Use case 3 focuses on containers attacking the host system, where a container exploits kernel vulnerabilities to gain control of the host machine, potentially impacting all containers running.

<sup>4</sup> <https://www.baeldung.com/linux/cgroups-and-namespaces>

- Use case 4 focuses on hosts attacking containers. Containers depend on the host for access to hardware, storage, memory, and network devices, which gives the host significant control over containerized processes.

In environments where host systems may not be fully trusted, like public clouds or Container-as-a-Service(CaaS), this creates a significant security concern. A semi-honest host can passively monitor container activities, such as system resource usage, process behaviors, and internal application states. Since the host controls the underlying infrastructure, it can observe the container process without breaking isolation mechanisms.

## 2.3 Monitoring

Monitoring is essential for tracking activity in containerized environments. This section first introduces Docker and its built-in monitoring tools, which provide crucial visibility into container states and running processes. It then examines system call tracing with `strace`, a technique central to this work that captures interactions between containerized applications and the operating system in real-time, including executed commands and stdin interactions that would otherwise be inaccessible.

### 2.3.1 Docker

Docker is an open platform for developing, shipping, and running applications using containerization technology. Containers allow applications to run in isolated environments while sharing the host machine's kernel, making them efficient and portable compared to virtual machines [4]. Docker packages an application and everything it needs to run into a portable unit called a container, allowing the application to behave consistently across different environments [4]. The heart of Docker is the Docker Engine, a system that manages and runs these containers. It includes a background process called the daemon, a REST API for interacting with the daemon, and a Command-Line Interface (CLI) for executing container operations [4]. The Docker REST API is a necessary component for monitoring because it enables external tools to retrieve real-time information about containers, such as process creation, termination, and state changes [4].

Docker is well-suited for managing dynamic workloads because it can optimize host resource usage. Docker images that serve as container creation templates streamline application distribution using a layered structure where only the modified components are updated [4]. By combining isolation, resource efficiency, and an accessible API, Docker provides a solid foundation for container lifecycle management and monitoring [4].

### 2.3.2 Docker Monitoring Tools

Docker provides commands for monitoring containers, allowing users to observe their internal activity. One such command is `docker events`, which reports changes in container states, such as when containers are created, started, or stopped. This is valuable for tracking container lifecycles and ensuring awareness of when containers become active. Additionally, the `docker top` command lets users inspect processes running inside a container, displaying process IDs (PIDs), commands, and other relevant details. These commands provide insight into container behavior, facilitating effective monitoring, issue identification, and troubleshooting.<sup>5, 6</sup>

### 2.3.3 Process Monitoring with `strace`

While commands like `docker events` and `top` provide high-level insights into container activity, more detailed process monitoring can be achieved using tools like `strace`. `Strace` is a diagnostic and debugging tool for Linux that intercepts and logs system calls made by containerized processes, along with any signals they receive.<sup>7</sup> System calls, such as `execve` (execute a program) and `write` (write data to a file), represent requests from user space applications to the kernel. Since all containers on a single host share the same kernel, `strace` can monitor these system calls regardless of namespace isolation [14]. This capability allows administrators to observe process-level activity, troubleshoot issues, and analyze program behavior without directly accessing the container. For example, `strace` can reveal how a container interacts with system resources, making it particularly useful for debugging or discreet monitoring in production environments.<sup>7</sup>

## 2.4 Related Work

Monitoring and securing containerized environments have become critical focus areas in recent years. With the rise of lightweight virtualization technologies like Docker, the need to understand system-level behavior, track executed commands, and analyze logs in real time has grown significantly. Various studies and tools have addressed these challenges, focusing on system call monitoring, centralized log collection, and real-time analysis.

This section reviews work that has contributed to the monitoring and logging of containerized systems. The discussion begins with studies on system call monitoring, including tools like `strace`. Next, it explores research on centralized logging and visualization solutions, particularly those leveraging the Elastic Stack (Elasticsearch, Logstash, and Kibana) to manage and analyze container logs. By examining these

---

<sup>5</sup><https://docs.docker.com/reference/cli/docker/system/events/>

<sup>6</sup><https://docs.docker.com/reference/cli/docker/container/top/>

<sup>7</sup><https://www.man7.org/linux/man-pages/man1/strace.1.html>

works, this review highlights existing approaches, their limitations, and how they relate to the goals of this study.

### 2.4.1 System Call Monitoring

System calls are the primary interface between user-space applications and the operating system kernel, enabling process management, file access, and resource control. Given their fundamental role, monitoring system calls has emerged as a key strategy for identifying and mitigating malicious activities in traditional and containerized environments [11].

As highlighted by [2], system calls represent a significant attack surface due to their role in core system operations, such as process management, file manipulation, and resource access. Table 2.4.1 shows high-risk system calls like `chmod`, `chown`, `execve`, and `mount`, which can be exploited to modify critical files or execute unauthorized code.

Table 2.4.1: Threat level 1 system calls

| System Calls                                                   | Dangerous Parameter                      |
|----------------------------------------------------------------|------------------------------------------|
| <code>chmod</code> , <code>fchmod</code>                       | a system file or a directory             |
| <code>chown</code> , <code>fchown</code> , <code>lchown</code> | a system file or a directory             |
| <code>execve</code>                                            | an executable file                       |
| <code>mount</code>                                             | on a system directory                    |
| <code>rename</code> , <code>open</code>                        | a system file                            |
| <code>link</code> , <code>symlink</code> , <code>unlink</code> | a system file                            |
| <code>create_module</code>                                     | modules not in <code>/lib/modules</code> |

While [2] emphasizes the importance of system calls as potential attack surfaces, particularly calls like `execve`, our focus extends to real-time monitoring of these calls for detecting unauthorized process executions.

### 2.4.2 Tools for System Call Monitoring

Tracking system calls can be achieved through several techniques, including the use of tools such as `strace`. Tools such as `strace` are used to intercept and log system calls. In [9], they compare `strace` to modern tools like `sysdig` and `DIO`, demonstrating its ability to capture syscall-level information and reduce performance cost, though with a higher performance overhead. In [5], the authors demonstrate how `strace` remains effective for tracing system calls and enabling observability of I/O behavior despite its performance trade-offs. Their analysis validates that `strace` can still serve as a practical tool for system call tracing, especially when detailed syscall-level information is required for diagnosing behavior or identifying security gaps.

### 2.4.3 Centralized Logging and Visualization

Beyond system call tracing, centralized logging and visualization play an important part in understanding and analyzing system behaviors. In [8], the author presents a logging solution using the tools Filebeat and Auditbeat for logging and Elastic Stack(Elasticsearch, Logstash, and Kibana) to aggregate, analyze, and visualize logs from Dockerized environments. Their work focuses on collecting `stderr` and `stdout` logs using Filebeat and Auditbeat. While this approach effectively captures application output and system audit logs, it does not monitor system calls or interactive shell commands. In contrast, our work actively tracks `execve` system calls using `strace` to capture real-time command executions within containerized environments. We provide a more granular view of container behavior by integrating this syscall-level information with Elasticsearch and Kibana.

### 2.4.4 Previous Work compared to this study

While existing studies, such as [8], provide centralized log collection and visualization solutions using tools like Filebeat, Auditbeat, and the Elastic Stack, they primarily focus on capturing application-level logs such as `stderr`, `stdout` or high-level audit events. Similarly, tools like `sysdig` and `DIO` [9] address syscall tracing with performance efficiency but lack targeted monitoring of specific system calls like `execve`. Our work builds on these approaches by integrating container metadata retrieval and attaching tools like `strace` for precise monitoring of `execve` system calls. By correlating real-time process executions with container activity and visualizing the data in Elasticsearch and Kibana, we provide fine-grained insights into interactive shell activity. This combined approach addresses the need for low-level system call tracing and centralized log visualization in containerized environments, bridging gaps identified in existing solutions.

# Chapter 3

## Architecture and Design

This chapter describes the system architecture and design of the real-time monitoring framework. The design ensures that container activity can be monitored in a way that remains undetectable while maintaining minimal system overhead. The system has three key components: process monitoring, log storage, and visualization. Each component plays a distinct role in capturing, processing, and presenting container activity.

### 3.1 System Overview

The monitoring system is designed to track interactive shell activity in containers without requiring modifications inside the container. It achieves this by attaching to containerized processes externally and capturing executed commands in real-time. The architecture is built around three components, illustrated in Figure 3.1.1.

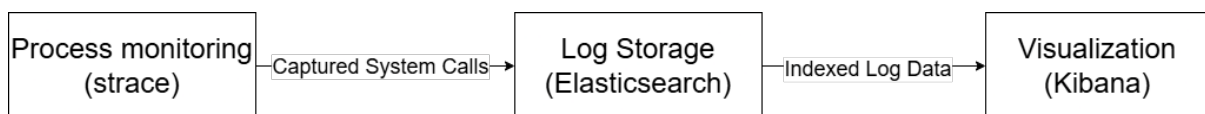


Figure 3.1.1: System architecture showing the three main components: Process Monitoring, Log Storage, and Visualization.

- **Process Monitoring:** Uses `strace` to trace system calls related to process execution inside containers.
- **Log Storage:** Stores collected execution data in Elasticsearch for efficient indexing and retrieval.
- **Visualization:** Uses Kibana to analyze and display monitored events in a structured format.

This modular architecture allows smooth monitoring while ensuring that logs remain hidden from containerized applications.

## 3.2 Component Breakdown

### 3.2.1 Process Monitoring

The first component is responsible for detecting and logging executed commands inside containers. The system uses `strace` to capture system calls related to command execution, specifically `execve`. By attaching `strace` externally, monitoring remains invisible to processes inside the container.

### 3.2.2 Log Storage

Captured command execution logs are stored in Elasticsearch. This component ensures structured indexing of collected data, allowing efficient search and retrieval. Logs include metadata such as container ID, process ID, executed command, and timestamp.

### 3.2.3 Visualization

Kibana is used to visualize container activity in real-time. It provides an interactive dashboard for analyzing monitored data, filtering logs based on specific criteria, and identifying trends in command execution.

## 3.3 Design Considerations

The system was designed with the following principles in mind:

- **Stealth:** The monitoring approach avoids modifying containers or injecting monitoring agents inside them.
- **Performance Efficiency:** The architecture minimizes performance overhead by limiting the amount of collected data.
- **Scalability:** The system is built to handle multiple monitored containers without significant degradation in performance.

## 3.4 Development Approach

A sprint-based development approach was used to refine the system architecture iteratively. The project was structured into weekly sprints, each focusing on a specific aspect of the design and implementation.

The early sprints focused on selecting the appropriate monitoring tools and validating their ability to capture relevant system calls without introducing excessive noise. Later sprints focused on improving log storage efficiency, structuring the collected data, and refining real-time visualization in Kibana.



This iterative process allowed continuous validation of architectural decisions. Adjustments were made based on performance testing, ensuring the final design met both stealth and efficiency requirements. The structured sprint approach also provided flexibility in testing alternative solutions when necessary.

### 3.5 Workflow

The monitoring process follows a structured workflow:

1. The event listener detects when a new container is created or started.
2. The system identifies interactive processes within the container.
3. `strace` is attached to these processes to capture executed commands.
4. The collected data is filtered, formatted, and sent to Elasticsearch.
5. Kibana provides a structured visualization of monitored commands.

This architecture ensures that container activity is tracked in real-time while remaining undetectable to the monitored system.

# Chapter 4

## Implementation

This chapter describes the monitoring tool developed to detect and trace interactive processes within containerized environments.

The implementation of the monitoring tool focuses on automating the detection of container lifecycle events, following processes such as `bash`, `sh`, and `ssh`, and exporting the collected data for storage and visualization. The tool is built to handle dynamic container environments efficiently, combining event detection, process tracing, and real-time data export.

Challenges like ensuring scalability through multithreading and filtering command executions to reduce redundancy are addressed throughout the implementation.

### 4.1 Workflow and Development Process

The monitoring tool was developed iteratively using a sprint-based approach. Each sprint introduced new functionality, allowing for incremental improvements and validation. Early development cycles focused on selecting an appropriate monitoring tool and establishing process detection mechanisms. Later stages integrated log storage and visualization, ensuring structured data collection.

Each sprint targeted specific milestones, ensuring that the tool evolved in a controlled and testable manner. The deliverables for each sprint are outlined in Table 4.1.1, detailing the systematic approach taken to build, refine, and evaluate the monitoring system.

Table 4.1.1: Overview of Sprints and Goals

| <b>Sprint</b>   | <b>Goals</b>                                          | <b>Deliverables</b>                                                       |
|-----------------|-------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Sprint 1</b> | Detect interactive processes and attach strace        | Event listener detects processes, executes strace for command tracing.    |
| <b>Sprint 2</b> | Export data to database and evaluate user ID tracking | Data export (timestamp, container, and command details) to Elasticsearch. |
| <b>Sprint 3</b> | Refine data export and visualize in Kibana            | Enhanced Elasticsearch export and basic Kibana visualization.             |
| <b>Sprint 4</b> | Visualize real-time data and test performance         | Performance testing (CPU, memory, I/O) and Kibana real-time dashboard.    |
| <b>Sprint 5</b> | Include user ID in data export                        | User ID was added to data export.                                         |

Each sprint cycle involved testing and validation, ensuring that newly implemented features met expectations. This iterative process enabled smooth integration between monitoring, data storage, and visualization components.

## 4.2 Event Detection and Monitoring

The monitoring tool begins by detecting container lifecycle events in real-time. Containers in modern environments are dynamic, often created, started, and stopped rapidly. To ensure monitoring captures newly started containers, the system listens for events that indicate when a container transitions to an active state.

The implementation uses the Docker Events API, which streams real-time events about container activity, including `create`, `start`, and `stop` events. The system focuses on `create` and `start` events to target only active or newly created containers for process monitoring.

The `listen_for_docker_events` function continuously listens for Docker events. When it detects a relevant event, it extracts the container ID and name and starts a new thread to monitor the container's processes. The multithreaded design allows the system to monitor multiple containers simultaneously without blocking the event listener. Listing 4.1 shows the code implementation of the `listen_for_docker_events` function.

```
1 def listen_for_docker_events():
2     for event in client.events(decode=True):
3         if event['Type'] == 'container' and event['Action'] in ('create', '
start'):
4             container_id = event['id']
5             container_name = event.get("Actor", {}).get("Attributes", {}).
get("name", "Unknown")
6             if container_id not in processed_containers:
7                 processed_containers.add(container_id)
8                 Thread(target=monitor_interactive_shell, args=(container_id
, container_name), daemon=True).start()
```

Listing 4.1: Event detection implementation using Docker events API

The `client.events` method, provided by the Docker SDK, streams events in a raw byte format by default. Without the `decode=True` parameter, the events appear as byte strings, which are not directly usable. For example, the raw output from the docker events stream looks as shown in Listing 4.2.

```
1 b' {
2     "status": "start",
3     "id": "b1e13a8c2f2d",
4     "from": "ubuntu",
5     "Type": "container",
6     "Action": "start",
7     "Actor": {
8         "ID": "b1e13a8c2f2d",
9         "Attributes": {
10            "name": "my_container"
11        }
12    },
13    "time": 1710239432,
14    "timeNano": 1710239432648493029
15 }'
```

Listing 4.2: Example of raw byte output from Docker events

The raw byte output must be manually decoded into a structured format using tools such as `json.loads()`. However, by specifying `decode=True` in the `client.events` method, Docker automatically transforms the byte output into a structured Python dictionary. The decoded output can then be accessed as a standard Python dictionary, as shown in Listing 4.3.

The transformation from raw byte output to a decoded dictionary improved readability and allowed straightforward access to metadata fields, such as `Type`, `Action`, and `Actor.Attributes.name`.

The docker events stream may include events unrelated to container lifecycle activities, such as network or volume events. For example, events with `Type: network` and `Action: connect` are common but irrelevant in this context.

```
1 {  
2   "Type": "container",  
3   "Action": "start",  
4   "id": "b1e13a8c2f2d",  
5   "Actor": {  
6     "ID": "b1e13a8c2f2d",  
7     "Attributes": {  
8       "name": "my_container"  
9     }  
10  },  
11  "time": 1710239432  
12 }
```

Listing 4.3: Decoded output of Docker events

To ensure efficiency, the system focuses only on container lifecycle events by filtering for `Type: container` with the `Action` values `create` and `start`. This approach ensures that only active or newly created containers are targeted for process monitoring. The function maintains a set of processed container IDs to avoid redundant processing and monitors each container only once, even if multiple events are generated. When a relevant event is detected, a new thread is immediately started to run the `monitor_interactive_shell` function. Using threads is critical for scalability and responsiveness, as sequentially monitoring processes would introduce delays and reduce performance. A multithreaded approach enables the system to monitor each container independently while the main thread remains free to listen for new events.

Although the listener attaches only to containers upon creation or startup, excluding pre-existing ones, the design remains scalable for dynamic environments. By monitoring each newly created container in parallel, the system avoids blocking operations and responds quickly to new events. This approach enables the system to continuously detect and track newly created containers, ensuring real-time monitoring as they become active.

## 4.3 Retrieving and Filtering Container Processes

Once active containers are detected, the tool proceeds to inspect their processes to identify interactive shells, such as `bash`, `sh`, or `ssh`. This is achieved using the `docker top` command, which retrieves a list of processes running inside a specified container. This step connects the detection of containers with monitoring their activities, enabling the tool to identify relevant processes for tracing while excluding meaningless ones. By default, the `docker top` command generates detailed process information, including User ID (UID), Process ID (PID), Parent process ID (PPID), and Command (CMD), as shown in Listing 4.4.

```
1 UID      PID    PPID    C  STIME TTY      TIME    CMD
2 root      1234   567     0  08:20 ?        00:00:00 /bin/bash
3 root      1256   567     0  08:21 ?        00:00:00 sleep 10
```

Listing 4.4: Default `docker top` output

Since many fields in the default `docker top` output are unnecessary for this tool's purposes, the command is refined with the flags `-eo pid,comm` to focus on the PID and COMMAND fields. The filtered result is shown in Listing 4.5. The UID in Listing 4.4 represents the process owner's user ID within the container, typically `root`. However, to identify the UID of the user initiating a command interactively, additional steps are required, as detailed in Section 4.3.2.

```
1 PID      COMMAND
2 1234      bash
3 1256      sleep
```

Listing 4.5: Filtered `docker top` output using `-eo pid,comm`

The `get_container_details` function implements the filtering by executing the refined `docker top` command and parsing its output. It extracts relevant processes by reading each line, skipping the header, and returning a list of tuples containing the container ID, process ID, and command name. This implementation is shown in Listing 4.6.

```
1 def get_container_details(container_id):
2     try:
3         top_output = subprocess.check_output(
4             ["docker", "top", container_id, "-eo", "pid,comm"]
5         ).decode("utf-8")
6         processes = top_output.splitlines()[1:]
7
8         details = []
9         for process in processes:
10             parts = process.split()
11             pid, cmd = parts[0], parts[1]
12             if cmd in ["sh", "bash", "ssh", "sshd"]:
13                 details.append((container_id, pid, cmd))
14         return details
15     except subprocess.CalledProcessError:
16         print(f"Container {container_id} is no longer running. Skipping...")
17     return []
```

Listing 4.6: Implementation of `docker top` in `get_container_details`

By filtering and extracting relevant processes, the system prepares for the next step of attaching monitoring tools like `strace` to these processes. This means that only meaningful activities within the container are traced, improving the monitoring tool's precision and efficiency.

### 4.3.1 System Call Tracing with strace

After identifying the relevant processes within the containers, the next step is to trace their system calls to collect detailed information about the commands executed. This step is important because tracing allows the tool to attach to the process and monitor its behavior in real-time. Without this, it would not be possible to capture `execve` system calls, which are triggered whenever a new program is executed. These system calls are essential for identifying and logging command executions within the container. To achieve this, `strace`, a utility specifically designed to monitor system calls, was integrated. In this implementation, `strace` is configured to focus on `execve` system calls to provide a comprehensive record of commands executed in the interactive processes within containerized environments.

The configuration of `strace` involves using specific flags and commands to tailor its output for the tool's purposes. The `stdbuf -oL` command ensures line-buffered output, allowing the tool to process `strace` logs immediately instead of waiting for the output buffer to fill. The `-f` flag allows `strace` to follow child processes, which is necessary since interactive shells often spawn additional processes. The `-s99999` flag ensures that command arguments are captured entirely, avoiding truncation of longer strings. The `-e trace=execve` flag limits the tracing to only the `execve` system calls, reducing the volume of unrelated data. Lastly, the `-ff` flag directs `strace` to separate the outputs of traced processes, making it easier to process multiple data streams. These commands and flags are executed together as part of a single command to ensure that `strace` collects all relevant information while minimizing extra details, enabling efficient monitoring of containerized processes.

The `strace` utility is executed within the `start_strace_on_pid` function, which uses the `subprocess` module to attach `strace` to the relevant process IDs identified during monitoring. This function initiates `strace` as a subprocess and ensures its output is captured and processed in real-time. The implementation of this function is shown in Listing 4.7.

```
1 def start_strace_on_pid(container_id, pid, container_name):
2     strace_cmd = f "sudo stdbuf -oL strace -f -p {pid} -s99999 -ff -e trace"
3     process = subprocess.Popen(
4         strace_cmd, shell=True, executable="/bin/bash", stdout=subprocess.
5         PIPE, text=True
6     )
```

Listing 4.7: Execution of `strace` in `start_strace_on_pid`

```
1 def filter_command(command, pid):
2     global recently_logged_commands
3     current_time = time.time()
4
5     excluded_commands = ["groups,", "dircolors,", "grep,", "grepconf.sh", "
6 locale,", "sed"]
7     if any(command.startswith(ex_cmd) for ex_cmd in excluded_commands):
8         return None
9
10    if command.startswith("ls"):
11        command = command.replace(" --color=auto", "")
12
13    if pid in recently_logged_commands:
14        last_command, last_time = recently_logged_commands[pid]
15        if command == last_command and (current_time - last_time) <
16        LOGGING_THRESHOLD_SECONDS:
17            return None
18
19    recently_logged_commands[pid] = (command, current_time)
20    return command
```

Listing 4.8: Filtering commands in `filter_command`

As `strace` runs, its output is monitored to detect and process `execve` system calls. Each `execve` call provides the program path and arguments, which are reconstructed into an entire command string. For example, a command like `ls -l /home` executed within a container is captured by `strace`, and the tool processes this information into a format suitable for logging. The filtering process is implemented in the `filter_command` function, as shown in Listing 4.8. This function checks commands against a list of excluded patterns, removes redundant flags (e.g., `--color=auto` in the case of `ls`), and prevents duplicate entries from being logged within a short time window defined by the `LOGGING_THRESHOLD_SECONDS` variable.

Capturing command executions in containerized environments presents unique challenges. Because processes often spawn multiple subprocesses, the desired commands frequently run within these subprocesses. To overcome this, the implementation tracks each `execve` call and then waits for the corresponding `SIGCHLD` signal from `strace`. The signal delivers information about the process and its UID. This approach ensures that all relevant commands, including those in subprocesses, are accurately captured, thus maintaining the completeness and reliability of the collected data. By filtering irrelevant commands, the tool focuses on meaningful activities, reducing log noise and improving the quality of data sent to Elasticsearch for analysis. This technique is beneficial for identifying user-driven actions rather than system-generated processes. A single user invocation of `ls` can lead to multiple `execve` calls with different PIDs due to shell wrappers, aliases, or subprocesses. Listing 4.9 illustrates unfiltered logs, where one user action leads to duplicate `ls` entries and unnecessary `execve` commands like `dircolors`, `groups`, `locale`, and `sed`, and filtered logs, where duplicates and irrelevant commands are removed.



```

1 # Unfiltered logs
2 [pid 1234] execve("groups", ["groups"], ...) = 0
3 [pid 1235] execve("dircolors", ["dircolors"], ...) = 0
4 [pid 1236] execve("grep", ["grep", "--color=auto"], ...) = 0
5 [pid 1237] execve("ls", ["ls", "--color=auto"], ...) = 0
6 [pid 1238] execve("ls", ["ls", "--color=auto"], ...) = 0
7 [pid 1239] execve("locale", ["locale"], ...) = 0
8 [pid 1240] execve("sed", ["sed"], ...) = 0
9
10 # Filtered logs
11 [pid 1237] execve("ls", ["ls"], ...) = 0

```

Listing 4.9: Unfiltered and filtered command logs

### 4.3.2 Buffering execve Calls and Resolving UIDs

Up to this point, the tool could detect container events, identify interactive processes, and monitor system calls using `strace`. However, the sequence of events observed in `strace` output differs depending on the type of command executed. For commands like `ls`, which do not spawn other processes, the `SIGCHLD` event appears immediately after the `execve` system call and `exited` event.

This immediate appearance allows the tool to capture the UID and finalize the command data promptly. Commands such as `su root` or `sh` do not trigger a `SIGCHLD` event immediately after the `execve` call. These commands often spawn a new shell, causing the `SIGCHLD` event to only appear when the shell exits, such as when the user logs out or terminates the session. In comparison, commands that do not spawn a new shell trigger the `SIGCHLD` event immediately upon termination. The following example (see Listing 4.10) illustrates these differences in `strace` output.

```

1 # Command with immediate SIGCHLD (e.g., ls)
2 [pid 1234] execve("/bin/ls", ["ls"], 0x7fff39c00d80 /* 10 vars */) = 0
3 [pid 1234] +++ exited with 0 +++
4 --- SIGCHLD {si_pid=1234, si_uid=1000, si_status=0, ...} ---
5
6 # Command that spawns a shell (e.g., su root)
7 [pid 2234] execve("/bin/su", ["su", "root"], 0x7fff39c00d80 /* 10 vars */)
8 = 0
9 # Shell remains active until the user logs out or terminates it

```

Listing 4.10: Comparison of commands with immediate and delayed `SIGCHLD` in `strace`

Because these two scenarios differ significantly, one providing an immediate `SIGCHLD` and the other delaying it, the buffering mechanism was designed to handle both. For commands that produce an immediate `SIGCHLD`, the tool updates the `execve_buffer` promptly with the UID and sends the finalized data to Elasticsearch. To handle cases where the `SIGCHLD` event is delayed, the tool buffers commands until the event occurs, ensuring no information is lost. Listing 4.11 illustrates the initialization of

`execve_buffer` and its role in tracking detected system calls while awaiting additional data.

```
1 execve_buffer = {}
2 exited_flag = False
3
4 # Buffer and track execve calls with provisional UIDs
5 match_execve = re.match(r'\[pid (\d+)\] execve\("[^"]+\)", \[(.*?)\]', line
6 )
7 if match_execve:
8     exec_pid = match_execve.group(1)
9     command_path = match_execve.group(2)
10    command_name = os.path.basename(command_path)
11    arguments_raw = match_execve.group(3)
12    arguments = re.findall(r'"([~"]+)"', arguments_raw)
13
14    if arguments and arguments[0] == command_name:
15        arguments = arguments[1:]
16
17    full_command = ' '.join([command_name] + arguments).strip()
18    commands = [cmd.strip() for cmd in full_command.split(",")]
19
20    for cmd in commands:
21        execve_data = {
22            "container_id": container_id,
23            "container_name": container_name,
24            "pid": exec_pid,
25            "command": [cmd],
26            "uid": "unknown",
27            "timestamp": datetime.datetime.now(
28                pytz.timezone("Europe/Stockholm")
29            ).isoformat()
30        }
31        if exec_pid in execve_buffer:
32            execve_buffer[exec_pid]["command"].append(cmd)
33        else:
34            execve_buffer[exec_pid] = execve_data
35    print(f"Execve Detected (Pending UID): {execve_buffer[exec_pid]}")
```

Listing 4.11: Initializing `execve_buffer` and buffering detected commands

When a process exits, the presence or absence of a `SIGCHLD` event determines how the tool finalizes the data. If a `SIGCHLD` signal is detected, the UID is extracted and added to the buffered command data. The finalized data is then sent to Elasticsearch, as illustrated below (see Listing 4.12).

This approach ensures that commands producing immediate `SIGCHLD` events are processed efficiently, while commands with delayed `SIGCHLD` events remain buffered until the required data becomes available. This robust mechanism provides accurate and reliable logging of command execution in containerized environments.

```

1 match_exited = re.match(r'\[pid (\d+)\] \+\+\+ exited', line)
2 if match_exited:
3     exited_pid = match_exited.group(1)
4     if exited_pid in execve_buffer:
5         print(f"Exited Detected for PID: {exited_pid}")
6         exited_flag = True
7
8 if exited_flag:
9     match_sigchld = re.match(r'.*--- SIGCHLD .*si_uid=(\d+)', line)
10    if match_sigchld:
11        si_uid = match_sigchld.group(1)
12        exited_flag = False
13        if exited_pid in execve_buffer:
14            execve_data = execve_buffer.pop(exited_pid)
15            execve_data["uid"] = si_uid
16            for cmd in execve_data["command"]:
17                data_with_uid = execve_data.copy()
18                data_with_uid["command"] = [cmd]
19                print(f"Updated Execve Data with UID: {data_with_uid}")
20                send_to_elasticsearch(data_with_uid)

```

Listing 4.12: Resolving UIDs and finalizing command details

## 4.4 Elasticsearch and Kibana Integration

Elasticsearch and Kibana were chosen for this implementation due to their powerful log management and visualization capabilities. Elasticsearch is the central backend for indexing and storing logs, while Kibana provides a user-friendly interface for analyzing and visualizing the data. Both tools were downloaded and configured directly on the host machine to ensure control, security, and smooth integration with the monitoring setup.

The monitoring system formats captured data into JSON objects before sending them to Elasticsearch. Each log entry includes metadata such as timestamp, container\_name, pid, command, and uid. These logs are structured to facilitate efficient querying and analysis. Elasticsearch indexes the received logs, making them retrievable in real-time.

In the `elasticsearch.yml` configuration, `network.host` is set to `localhost`, restricting Elasticsearch access exclusively to the host. This prevents other containers in the same environment from discovering or interacting with the monitoring system. Logs can also be accessed using the command shown in Listing 4.13.

```

1 curl -X GET "localhost:9200/container_logs/_search?size=1000&_source=
    timestamp,container_name,pid,command,user_id&pretty"

```

Listing 4.13: Retrieving logs directly from Elasticsearch

Although not explicitly designed as a fallback mechanism, this method remains reliable for verifying log indexing and retrieving stored data if the Kibana interface

is unavailable. It is also helpful for troubleshooting or confirming system functionality directly from within the host.

Kibana is configured through `kibana.yml`, with `server.host` set to `localhost`, which limits access to the virtual machine itself. Since Elasticsearch is also restricted to `localhost`, both components remain hidden from external systems, including containers within the same network. As a result, only the host system can interact with the monitoring interface, allowing container activity to be observed discreetly.

Kibana depends on Elasticsearch to retrieve data, meaning its availability is directly linked to Elasticsearch's configuration. To organize and display the collected information, an index pattern named `container_logs*` is defined in Kibana. This setup allows key fields such as `timestamp`, `container_name`, `pid`, `command`, and `uid` to be accessed and analyzed.

By integrating Elasticsearch and Kibana, the monitoring system enables real-time observation of container activity while ensuring data remains structured and accessible. This configuration supports long-term storage and dynamic querying, providing an efficient logging solution for monitored environments.

This setup ensures both security and operational stealth. By restricting Elasticsearch and Kibana exclusively to the host, no external system, including other containers, can discover or interact with them. Should the Kibana interface become inaccessible, direct Elasticsearch queries via `curl` provide an alternative means of retrieving monitoring data, preserving access without compromising invisibility.

# Chapter 5

## Evaluation and Result

This chapter presents a detailed evaluation of the monitoring tool's performance in a controlled containerized environment. The infrastructure and environment used for testing are briefly described, and their impact on the results is considered. The chapter then compares system performance with and without logging enabled, focusing on CPU, memory, and I/O usage. The collected log data is analyzed in terms of its structure, usability, and how well it integrates with Elasticsearch and Kibana. Various outputs will be presented, including container event logs, Elasticsearch entries, Kibana visualizations, and the captured `execve` system call output from within a monitored container. These outputs provide insight into the monitoring process and its effectiveness in capturing interactive shell activity. Visualizations highlight the resource overhead introduced by the monitoring process, clearly comparing its impact.

### 5.1 Infrastructure setup

The experiments were conducted in a virtualized environment accessed via SSH. The virtual machine (VM) was the testbed for deploying and monitoring containerized workloads. A VPN connection managed through Tunnelblick, was used to access the VM remotely. The monitoring tool was tested in a standalone containerized environment. It listened for container lifecycle events, attached to interactive processes, and traced system calls using `strace`, exporting relevant data to Elasticsearch. The tool's integration with Elasticsearch and Kibana enabled real-time log collection and visualization, allowing for structured analysis of container activity. Elasticsearch and Kibana were installed on the host machine to store and visualize logs. Elasticsearch was configured with `network.host` set to `localhost`, which allowed access only from within the VM. During testing, Kibana's `server.host` was temporarily set to `0.0.0.0` to enable access from the host browser. In a secure deployment, however, `server.host` should be set to `localhost` to prevent external access and ensure that the interface remains undetectable by other containers or systems.

To simulate real-world container activity, a script named `testrun.sh` was used, automating the creation, execution, and termination of 100 containers, each running an interactive shell. The script first deployed the containers based on the `app1-image`, assigning each a unique SSH port. Once the containers were active, it simulated user activity by executing basic commands, such as listing files recursively and writing to disk, creating a workload that mimicked real interactions within a containerized environment. After completing the execution phase, the script verified access to each container and logged timestamps before terminating all containers, ensuring that each test started from a clean state. The script created a controlled but dynamic environment where the monitoring tool could detect, analyze, and log container behavior in real-time by generating ephemeral workloads and interactive shell activity. This setup enabled a structured system performance evaluation, ensuring that results remained reproducible when comparing resource usage with and without monitoring enabled.

## 5.2 Validation of Log Retrieval

To validate the correctness of the collected logs, the ‘curl’ command previously described in Listing 4.13 was executed to retrieve container activity records directly from Elasticsearch. This method ensures that logged commands are correctly stored and accessible even in the absence of Kibana. The query successfully retrieved structured log data, confirming the system’s ability to track interactive shell activity and executed processes within monitored containers. As shown in Figure 5.2.1, the retrieved logs contain essential metadata, including the container name, process ID, executed command, and timestamp.

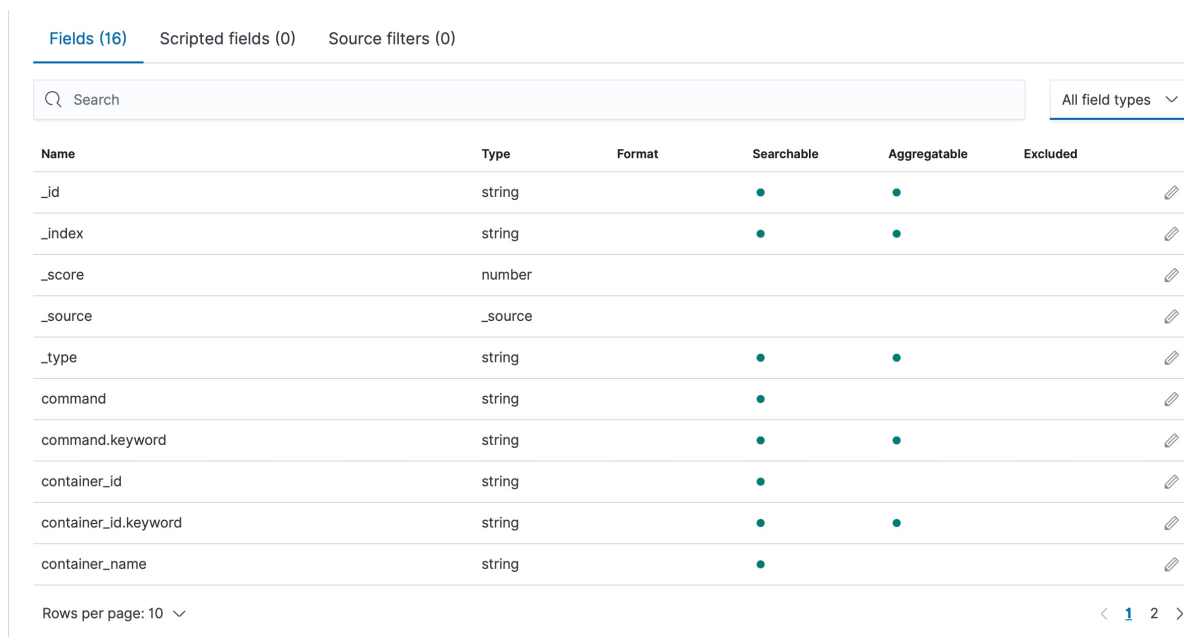
```
{
  "_index" : "container_logs",
  "_type" : "_doc",
  "_id" : "z2_XwZQBy5G4q29qFJ9i",
  "_score" : null,
  "_source" : {
    "container_name" : "app2",
    "pid" : "498260",
    "command" : [
      "sshd /usr/sbin/sshd -D -R"
    ],
    "timestamp" : "2025-02-01T15:08:02.131797+01:00"
  },
  "sort" : [
    1738418882131
  ]
},
```

Figure 5.2.1: Example output from direct Elasticsearch query showing logged container activity.

The retrieved output includes expected commands such as `sh -c cat /da`, `cat /hej`,

and `sshd /usr/sbin/sshd -D -R`, confirming that the monitoring system logs user interactions and process executions correctly. However, this raw output is less filtered and more challenging to interpret compared to the structured visualization in Kibana.

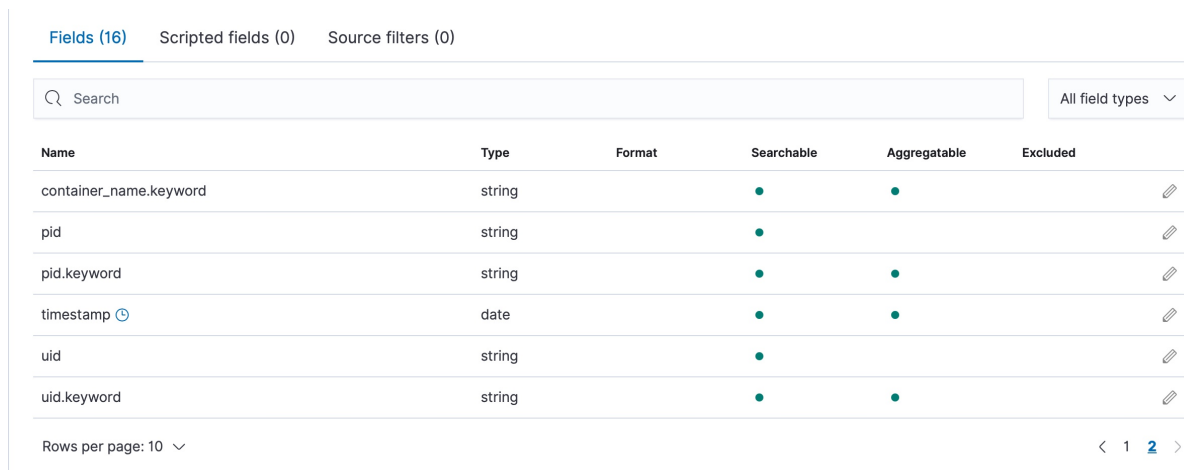
Kibana automatically detects fields from Elasticsearch, displaying them in a structured format (Figure 5.2.2 and Figure 5.2.3). The key fields relevant to the monitoring tool include `timestamp`, which ensures chronological sorting, `container_name`, which links each log entry to a specific container, and `pid`, which records the process ID of executed commands. The `command` field captures the exact command issued within the container, while `uid` provides information on the user executing the command. These fields are needed to analyze interactive shell activity and track user actions within the containerized environment.



The screenshot shows the Kibana Fields interface. At the top, there are tabs for 'Fields (16)', 'Scripted fields (0)', and 'Source filters (0)'. Below the tabs is a search bar and a dropdown for 'All field types'. The main table lists 16 fields with columns for Name, Type, Format, Searchable, Aggregatable, and Excluded. The fields are: `_id` (string), `_index` (string), `_score` (number), `_source` (string), `_type` (string), `command` (string), `command.keyword` (string), `container_id` (string), `container_id.keyword` (string), and `container_name` (string). The 'Rows per page' is set to 10, and the page number is 1.

| Name                              | Type                 | Format | Searchable | Aggregatable | Excluded |
|-----------------------------------|----------------------|--------|------------|--------------|----------|
| <code>_id</code>                  | string               |        | •          | •            |          |
| <code>_index</code>               | string               |        | •          | •            |          |
| <code>_score</code>               | number               |        |            |              |          |
| <code>_source</code>              | <code>_source</code> |        |            |              |          |
| <code>_type</code>                | string               |        | •          | •            |          |
| <code>command</code>              | string               |        | •          |              |          |
| <code>command.keyword</code>      | string               |        | •          | •            |          |
| <code>container_id</code>         | string               |        | •          |              |          |
| <code>container_id.keyword</code> | string               |        | •          | •            |          |
| <code>container_name</code>       | string               |        | •          |              |          |

Figure 5.2.2: Overview of detected fields in Kibana (first set of fields).



The screenshot shows the Kibana Fields interface with the second set of fields. The tabs and search bar are the same. The table lists 6 fields: `container_name.keyword` (string), `pid` (string), `pid.keyword` (string), `timestamp` (date), `uid` (string), and `uid.keyword` (string). The 'Rows per page' is set to 10, and the page number is 2.

| Name                                | Type   | Format | Searchable | Aggregatable | Excluded |
|-------------------------------------|--------|--------|------------|--------------|----------|
| <code>container_name.keyword</code> | string |        | •          | •            |          |
| <code>pid</code>                    | string |        | •          |              |          |
| <code>pid.keyword</code>            | string |        | •          | •            |          |
| <code>timestamp</code>              | date   |        | •          | •            |          |
| <code>uid</code>                    | string |        | •          |              |          |
| <code>uid.keyword</code>            | string |        | •          | •            |          |

Figure 5.2.3: Overview of detected fields in Kibana (second set of fields).

With the index pattern configured, the logs can be explored in Kibana's discovery view (Figure 5.2.4). The interface allows filtering based on specific fields, making it possible to track patterns in container activity over time. Each log entry contains a timestamp, container ID, process ID, and command, providing a detailed view of executed processes. The histogram visualization at the top of the interface highlights peaks in activity, indicating moments of increased command execution. The ability to search and filter logs based on fields such as uid enables fine-grained analysis of user interactions within containers.

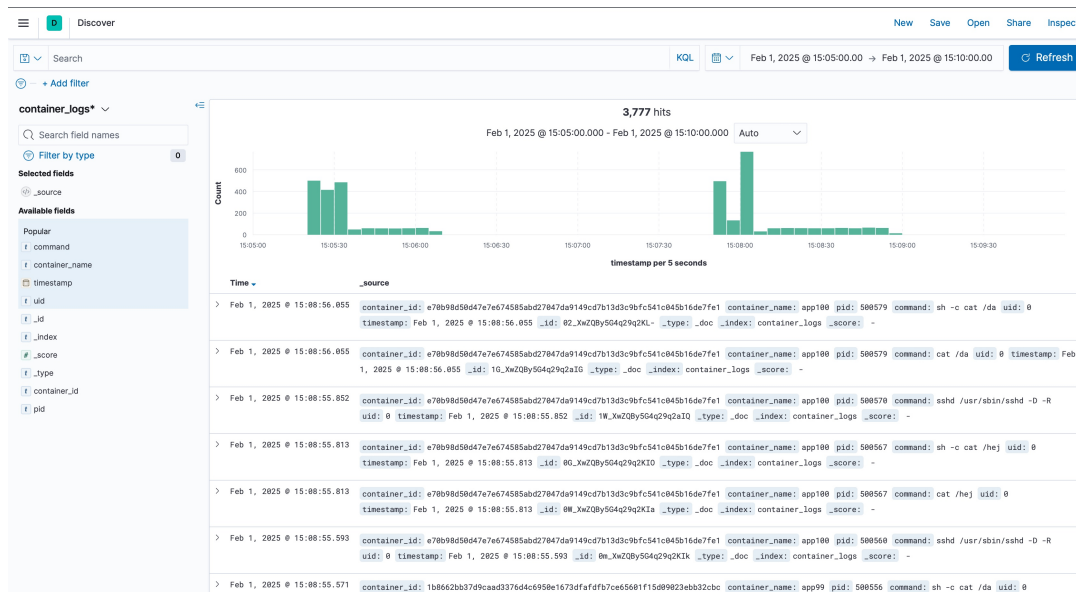


Figure 5.2.4: Kibana discovery view showing container log visualization.

The logs collected provide insight into real-time container activity, reflecting the execution of monitored commands. The stored data includes commands like `ls -R /` and `sh -c cat /da`, which were executed as part of the automated workload generated during testing. These logs validate the monitoring tool's effectiveness in capturing interactive shell activity and exporting structured data to Elasticsearch for further analysis.

By integrating Elasticsearch and Kibana, the collected logs are stored and made accessible for real-time visualization and analysis. This enables direct monitoring of command execution across multiple containers, providing a clear representation of system activity. The structured format of the logs ensures that each entry can be queried, filtered, and visualized efficiently, supporting both general monitoring and targeted analysis of specific user actions.



### 5.3 Comparison of Logging vs. Non-Logging

The performance impact of logging was evaluated by comparing system resource usage in two scenarios: with logging enabled and without logging. The collected data, visualized in Figure 5.3.1, illustrates CPU usage, memory consumption, and disk write operations over time.

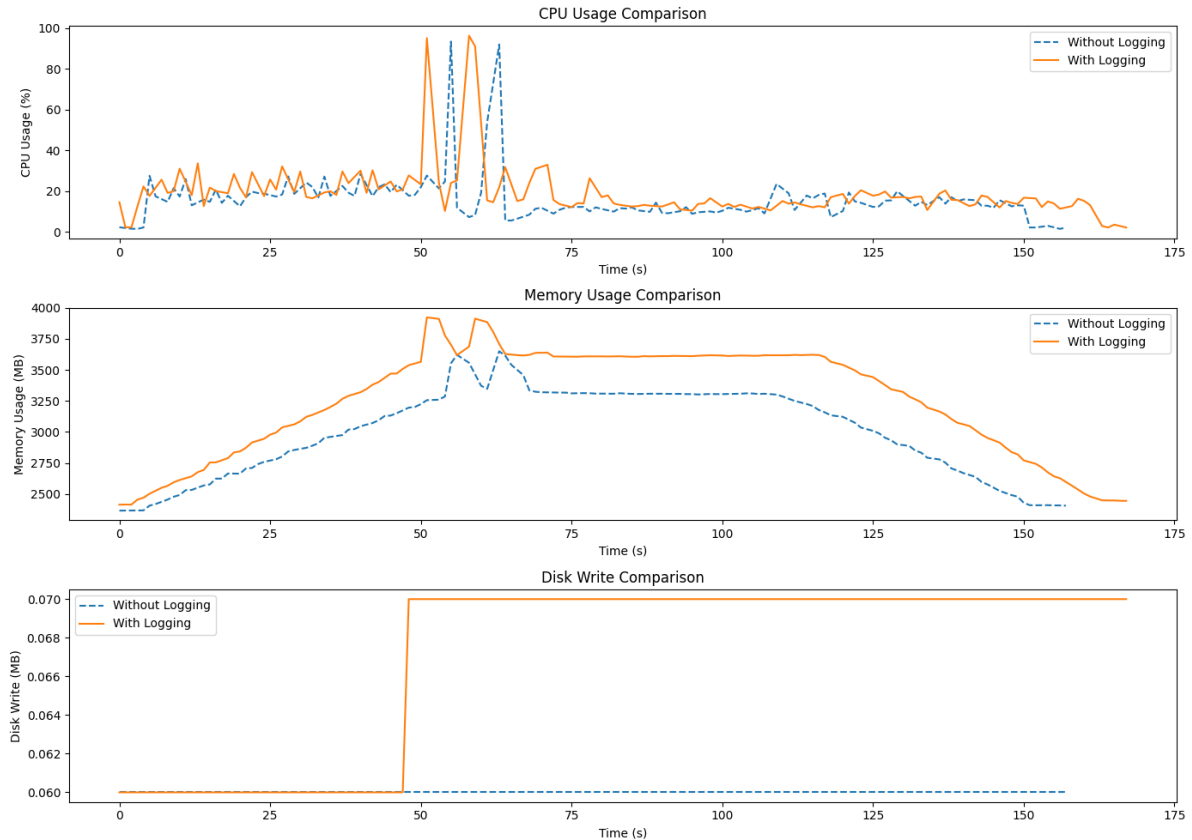


Figure 5.3.1: Comparison of system resource usage with and without logging enabled

The CPU usage graph in Figure 5.3.1 shows multiple spikes in both scenarios, indicating moments of increased container activity. These peaks likely correspond to periods where the script executes commands inside the containers, causing a temporary increase in processing load. However, the logging-enabled case exhibits a significantly higher number of spikes, suggesting that the monitoring tool introduces periodic bursts of CPU usage. This behavior is likely caused by the overhead of continuously attaching to new processes, capturing system calls, and sending logs to Elasticsearch.

The memory usage comparison highlights the increased resource demand introduced by the monitoring system. With logging enabled, memory consumption follows a similar trend to the non-logging case but remains consistently higher, peaking around 4,000 MB. This increase is due to the overhead introduced by attaching to processes and storing command execution data before sending it to Elasticsearch. The fluctuations in both cases suggest that memory usage is influenced by the

number of active containers at any given moment, with logging amplifying the overall consumption.

The disk write operations show a significant difference between the two scenarios. While disk activity remains minimal in the non-logging case, logging introduces a noticeable jump, where log data is continuously written and sent to Elasticsearch. The sharp increase in disk writes suggests that logs are either buffered before being committed or that a sudden rise in monitored activity results in more frequent writes.

These results confirm that logging introduces additional system overhead, particularly in CPU and memory usage, and leads to a higher frequency of CPU spikes. The increased number of spikes in the logging case suggests that the monitoring tool's operations, such as process attachment, syscall tracing, and log transmission, contribute to periodic CPU bursts. Despite this added overhead, the monitoring tool remains functional, capturing real-time command execution data without overwhelming system resources.

## **5.4 Limitations and Observations**

The monitoring system was tested with 100 containers in a controlled environment. Higher container counts were not tested, so its performance at larger scales remains unknown. While the multithreaded design improved efficiency, potential bottlenecks, and resource constraints could emerge under heavier workloads. Further testing is needed to assess stability and scalability.

Due to time constraints, performance evaluation focused on measuring system resource usage within a limited scope. Additionally, alternative log storage and visualization solutions were not explored, as Elasticsearch and Kibana were selected based on prior familiarity and proven reliability. Future research should investigate scalability under higher workloads and assess alternative monitoring architectures.

# Chapter 6

## Conclusion and Future Work

This chapter summarizes the key findings from evaluating the container monitoring system, reflecting on its effectiveness in achieving stealthy monitoring from the host machine. It also explores potential directions for future research, including optimizations for efficiency, improvements in filtering mechanisms, and broader applications of stealthy monitoring in containerized environments.

### 6.1 Conclusion

This thesis demonstrates that a host in a containerized environment can effectively monitor container activity without detection. By leveraging system call tracing techniques such as `strace`, executed commands can be captured in real-time while remaining invisible to containerized applications. The findings confirm that this approach functions reliably and introduces minimal overhead, making it a viable method for monitoring large-scale container deployments.

One of the main contributions of this research is proving that monitoring can be both stealthy and efficient. Traditional logging solutions often require modifications within the container or rely on exposed APIs, which can be detected or bypassed. This implementation circumvents those limitations by monitoring processes directly from the host, ensuring containers remain unaware of the monitoring activity. The integration of Elasticsearch for structured log storage and Kibana for visualization further enhances the usability of the collected data, providing a scalable way to analyze container activity.

The performance evaluation confirms that the approach remains effective under realistic conditions. The results demonstrate that while monitoring does introduce some CPU and memory overhead, it is within acceptable limits and does not significantly impact system performance. Scalability tests suggest the system can handle multiple containers in parallel, making it suitable for larger deployments.

While the implementation has proven successful, it also raises important ethical

considerations. The ability to monitor containers without detection introduces potential privacy concerns, especially in multi-tenant environments where different organizations share infrastructure. The study underscores the importance of deploying such monitoring techniques only in environments where the host has full administrative control and explicit authorization to conduct security monitoring. Ensuring transparency and compliance with security policies is crucial to prevent misuse.

Beyond real-time visibility, the collected data can be used to detect anomalies, such as unusual command patterns or unexpected spikes in activity. It also supports performance diagnostics by correlating command execution with system resource usage. Over time, this log data can provide valuable insights for forensic analysis, compliance audits, and proactive threat detection, making it a useful resource beyond its primary monitoring function.

Overall, this research contributes to the field of container security by demonstrating a practical, efficient, and scalable approach to stealthy monitoring. It provides a foundation for further exploration into advanced security observability techniques that do not compromise the integrity or performance of containerized applications.

## 6.2 Future Work

While this research has successfully demonstrated the feasibility of stealthy monitoring in containerized environments, several areas remain open for further exploration. One key direction involves optimizing performance to reduce the computational overhead further. While the system already operates efficiently, refining the tracing mechanism to capture only the most relevant system calls could improve efficiency, especially in high-load environments.

Scalability testing on a much larger scale is another critical area for future work. The current evaluation confirms that the approach remains viable under moderate workloads, but testing in environments with hundreds or thousands of active containers would provide deeper insights into potential bottlenecks. Investigating distributed monitoring techniques, such as spreading the tracing load across multiple host nodes, could enhance large-scale deployment performance.

Another promising area for research is improving log filtering mechanisms. While the current implementation effectively reduces redundant entries, further refinements could enhance the precision of captured data. For example, developing more sophisticated filtering rules to prioritize security-critical events while discarding low-value system calls could optimize data collection and storage.

Integrating additional detection capabilities is also a potential direction for future work. By correlating system call patterns with known attack behaviors, the monitoring system could be enhanced to identify suspicious activity beyond simple command execution. Machine learning-based anomaly detection could be explored to

automatically flag unusual container behavior, adding a proactive security layer to the monitoring system.

Finally, future work could explore broader and more advanced applications of the collected data. While the current system enables forensic analysis and compliance auditing, further research could enhance its analytical capabilities. For instance, integrating machine learning techniques for behavioral anomaly detection, correlating command patterns with user-specific actions, or supporting long-term trend analysis could significantly increase the system's value. Exploring alternative backend integrations, such as message queues or time-series databases, or extending the monitoring approach using eBPF for deeper visibility, could also open new avenues for scalable and high-fidelity observability.

By addressing these areas, future research can build upon the findings of this study to further refine stealthy monitoring techniques, enhance efficiency, and expand its applicability in containerized environments. Continued exploration in this field will contribute to more effective and scalable container observability and security solutions.

# Bibliography

- [1] Bentaleb, Ouafa, Belloum, Adam SZ, Sebaa, Abderrazak, and El-Maouhab, Aouaouche. “Containerization technologies: Taxonomies, applications and challenges”. In: *The Journal of Supercomputing* 78.1 (2022), pp. 1144–1181.
- [2] Bernaschi, Massimo, Gabrielli, Emanuele, and Mancini, Luigi V. “Operating system enhancements to prevent the misuse of system calls”. In: *Proceedings of the 7th ACM conference on Computer and communications security*. 2000, pp. 174–183.
- [3] Bhat, Sathyajith and Bhat, Sathyajith. “Introduction to containerization”. In: *Practical Docker with Python: Build, Release, and Distribute Your Python App with Docker* (2022), pp. 1–9.
- [4] Eftimie, Alexandru and Borcoci, Eugen. “Containerization Using Docker Technology”. In: ().
- [5] Esteves, Tânia, Macedo, Ricardo, Oliveira, Rui, and Paulo, João. *Diagnosing applications’ I/O behavior through system call observability*. 2023. arXiv: 2304.08569 [cs.DC]. URL: <https://arxiv.org/abs/2304.08569>.
- [6] GDPR, GDPR. “General data protection regulation”. In: *Regulation (EU) 679* (2016).
- [7] Li, Xiao-Feng. *Advanced design and implementation of virtual machines*. CRC Press, 2016.
- [8] Moilanen, Lauri. “Collecting Logs from Docker Containers”. In: (2020).
- [9] Nascimento, Rodrigo Pereira do and Lejbman, Alfredo Goldman vel. “The Tracer Files: Cracking the Case of Performance Impact in Tracing Linux File I/O for I/O-Intensive Applications”. In: *2024 International Symposium on Computer Architecture and High Performance Computing Workshops (SBAC-PADW)*. 2024, pp. 50–57. DOI: 10.1109/SBAC-PADW64858.2024.00018.
- [10] Nissenbaum, Helen. “A contextual approach to privacy online”. In: *Daedalus* 140.4 (2011), pp. 32–48.
- [11] Rajagopalan, M., Hiltunen, M.A., Jim, T., and Schlichting, R.D. “System Call Monitoring Using Authenticated System Calls”. In: *IEEE Transactions on Dependable and Secure Computing* 3.3 (2006), pp. 216–229. DOI: 10.1109/TDSC.2006.41.

- [12] Research, Grand View. *Application Container Market Size, Share & Trends Analysis Report*. Accessed: March 3, 2025. 2023. URL: <https://www.grandviewresearch.com/industry-analysis/application-container-market>.
- [13] Research, Verified Market. *Application Container Market Size, Trends, And Forecast*. Accessed: March 3, 2025. 2024. URL: [https://www.verifiedmarketresearch.com/product/application-container-market/?utm\\_source=chatgpt.com](https://www.verifiedmarketresearch.com/product/application-container-market/?utm_source=chatgpt.com).
- [14] Rice, Liz. *Container security: Fundamental technology concepts that protect containerized applications*. "O'Reilly Media, Inc.", 2020.
- [15] Singh, Saurabh, Jeong, Young-Sik, and Park, Jong Hyuk. "A survey on cloud computing security: Issues, threats, and solutions". In: *Journal of Network and Computer Applications* 75 (2016), pp. 200–222.
- [16] Smith, James E. *Virtual Machines: Versatile Platforms for Systems and Processes*. 2005.
- [17] Sultan, Sari, Ahmad, Imtiaz, and Dimitriou, Tassos. "Container Security: Issues, Challenges, and the Road Ahead". In: *IEEE Access* 7 (2019), pp. 52976–52996. DOI: 10.1109/ACCESS.2019.2911732.
- [18] Vethanayagam, Suhanthan. "Threat Identification from Access Logs Using Elastic Stack". In: (2020).